

k-vecinos más cercanos

Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales
Inteligencia Artificial
(TC3002B.M2)

José Carlos Ortiz Bayliss

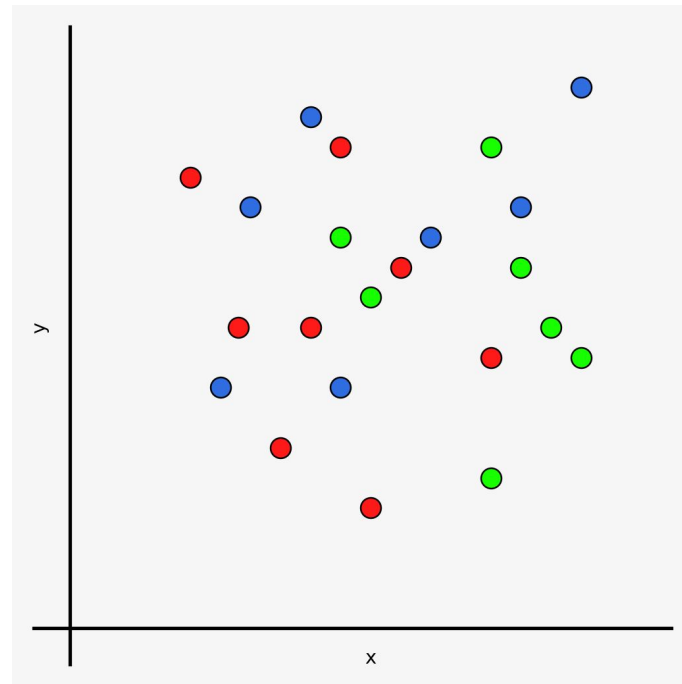


***k*-vecinos más cercanos**

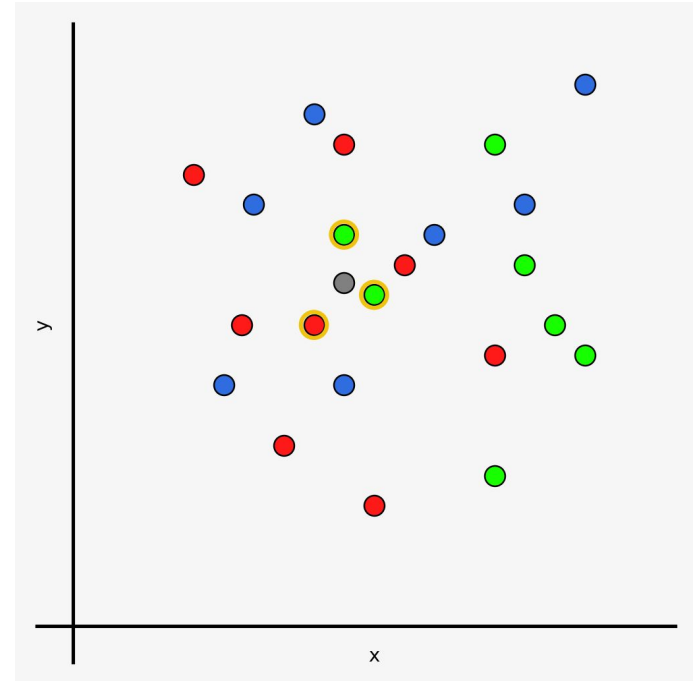
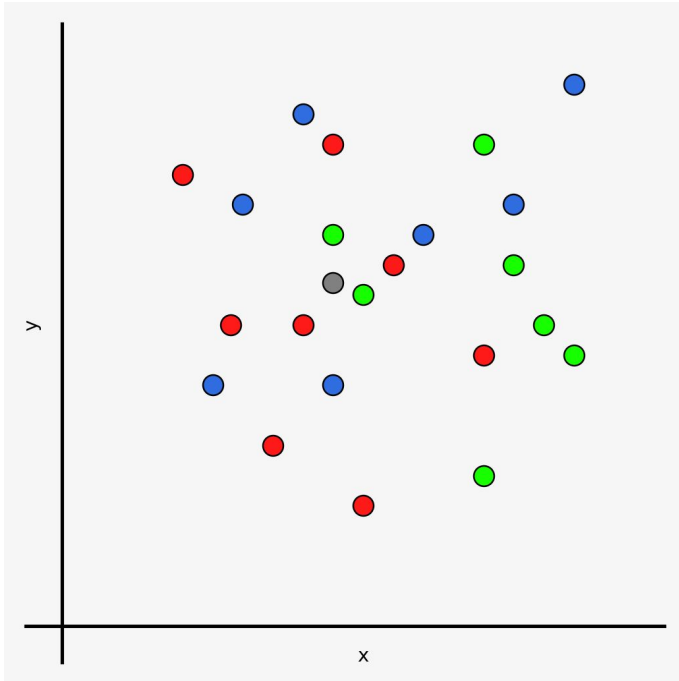
- *k*-vecinos más cercanos es un modelo de aprendizaje supervisado basado en la distancia.
- Cuando un caso debe clasificarse, se usa una medida de distancia para identificar los *k*-vecinos más cercanos. De esos vecinos, la clase que más aparece es la que se asigna al nuevo caso.
- A diferencia de otras técnicas, *k*-vecinos más cercanos no genera un modelo, sino que los datos de entrenamiento siempre son usados para clasificar nuevos casos.
- Debido a que el modelo trabaja con base en distancias, las variables independientes deben ser cuantitativas.

¿Cómo funciona el algoritmo de los k -vecinos más cercanos?

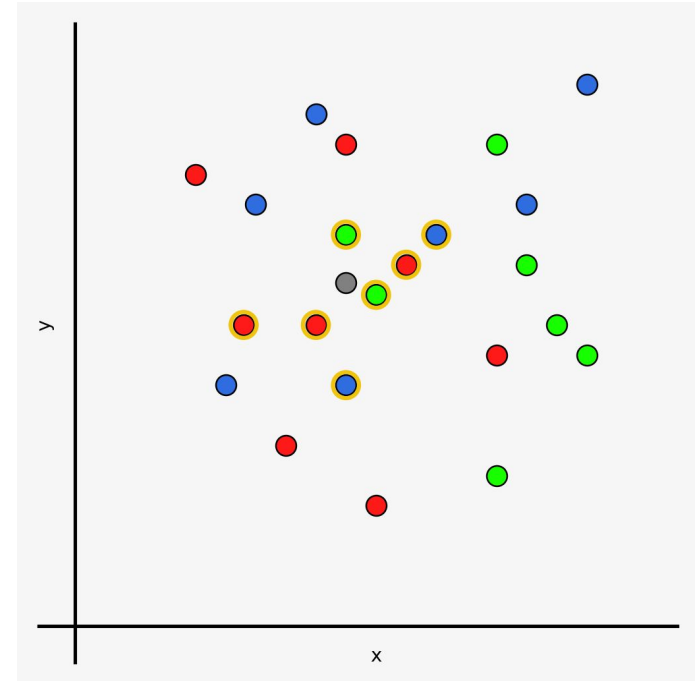
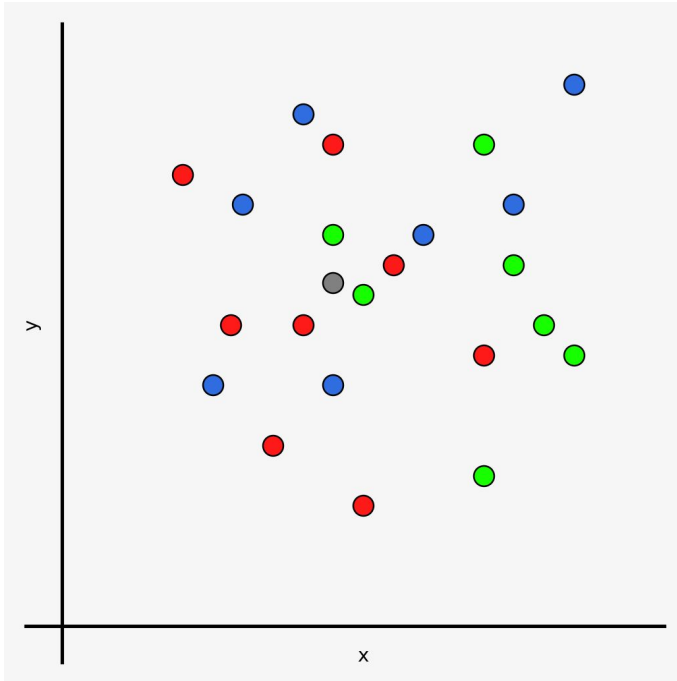
- El algoritmo compara cada nuevo caso con los ejemplos del conjunto de entrenamiento.
- Emplea una métrica de distancia que permite medir qué tan cercano es el nuevo dato a los ya conocidos.
- A partir de esta proximidad, se define un vecindario de puntos similares y se asigna al nuevo elemento la clase que predomina entre ellos.



¿Cómo funciona el algoritmo de los k -vecinos más cercanos?



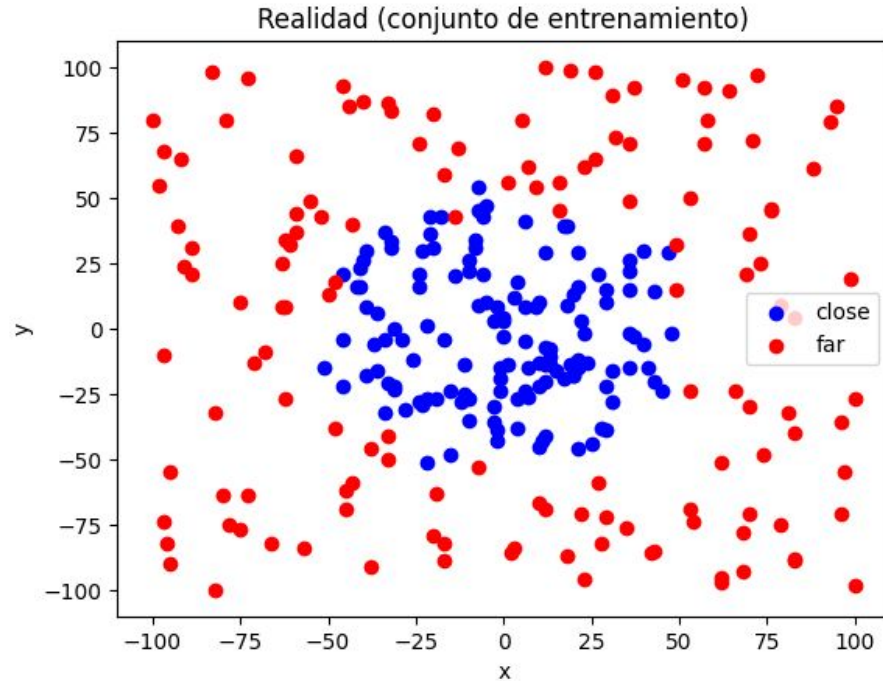
¿Cómo funciona el algoritmo de los k -vecinos más cercanos?



El tamaño adecuado para el vecindario

- El tamaño recomendado para k es la raíz cuadrada del número de ejemplos en el conjunto de entrenamiento.
- Si k es muy pequeña, las decisiones tienden a ser locales y ser susceptibles al ruido.
- Si, por el contrario, k es muy grande, las decisiones se ven afectadas por la estructura global del dataset.
- En el peor caso, si k es demasiado grande, el algoritmo se comportará como el clasificador ZeroR.

Clasificación con clases no linealmente separables



Clasificación usando k -vecinos más cercanos

```
# Importamos los módulos necesarios
import pandas as pd
from sklearn import neighbors
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Leemos los datos que usaremos
points = pd.read_csv("centered.csv")

# Separamos las variables independientes de la dependiente
X = points.drop("Class", axis = 1).values
y = points["Class"].values

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, stratify = y,
                                                    random_state = 54321)
```


Clasificación usando k -vecinos más cercanos

```
# Creamos un modelo de clasificación usando el algoritmo de k-vecinos más cercanos
model = neighbors.KNeighborsClassifier(3, weights = "uniform")

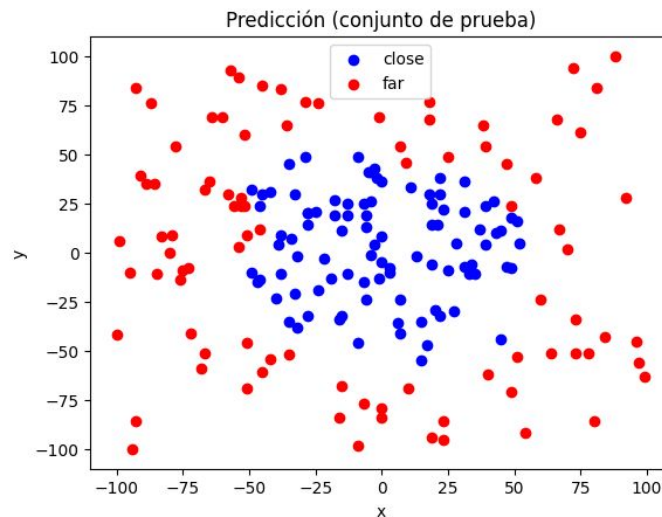
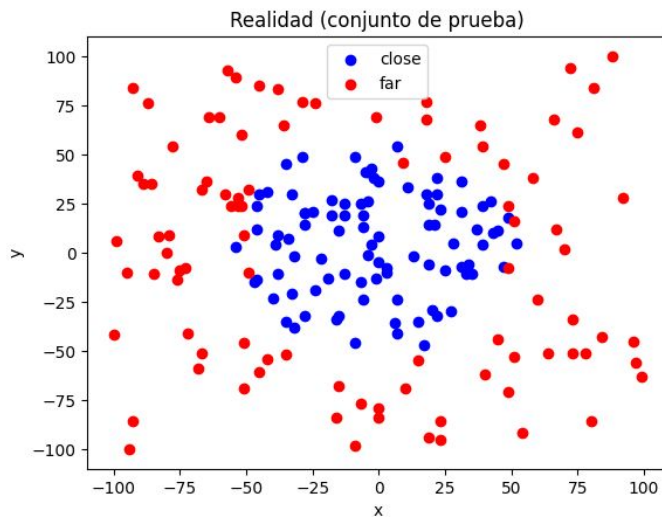
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

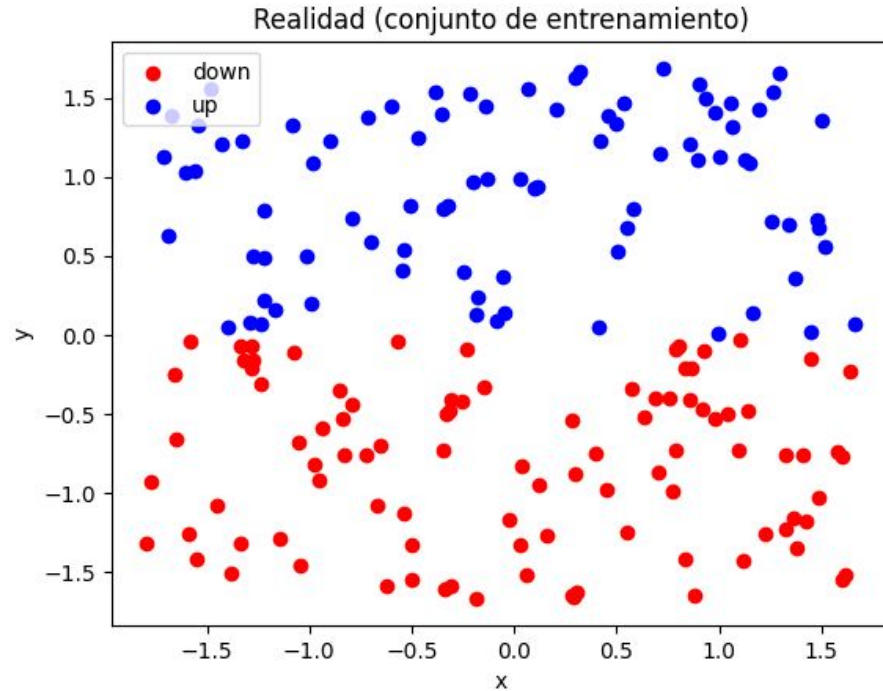
# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Evaluación del desempeño

		Predicción	
		close	far
Realidad	close	88	3
	far	6	86



Clasificación con variables independientes en rangos muy diferentes



Sensibilidad de kNN a la escala de las variables

```
# Importamos los módulos necesarios
import pandas as pd
from sklearn import neighbors
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Leemos los datos que usaremos
points = pd.read_csv("vertical.csv")

# Separamos las variables independientes de la dependiente
X = points.drop("Class", axis = 1).values
y = points["Class"].values

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, stratify = y,
                                                    random_state = 54321)
```

Sensibilidad de kNN a la escala de las variables

```
# Creamos un modelo de clasificación usando el algoritmo de k-vecinos más cercanos
model = neighbors.KNeighborsClassifier(5, weights = "uniform")

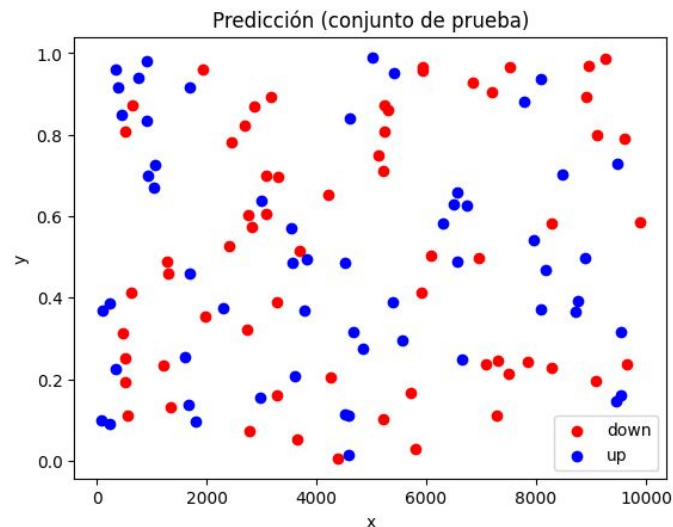
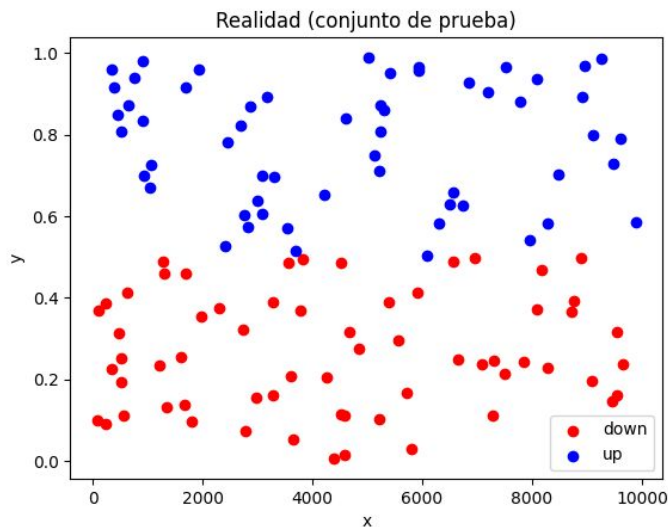
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Evaluación del desempeño

		Predicción	
		down	up
Realidad	down	30	33
	up	33	24



Escalamiento de las variables

```
# Importamos los módulos necesarios
import pandas as pd
from sklearn import neighbors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix

# Leemos los datos que usaremos
points = pd.read_csv("vertical.csv")

# Separamos las variables independientes de la dependiente
X = points.drop("Class", axis = 1)
y = points["Class"]

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, stratify = y,
                                                    random_state = 54321)
```

Escalamiento de las variables

```
# Importamos los módulos necesarios
import numpy as np
import pandas as pd
from sklearn import neighbors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix

# Creamos un escalador de datos
scaler = StandardScaler()

# Se ajusta el escalador usando únicamente los datos de entrenamiento
scaler.fit(X_train)

# Escalamos las variables independientes del entrenamiento y prueba
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```


Se genera el modelo usando los datos escalados

```
# Creamos un modelo de clasificación usando el algoritmo de k-vecinos más cercanos
model = neighbors.KNeighborsClassifier( 5, weights = "uniform")

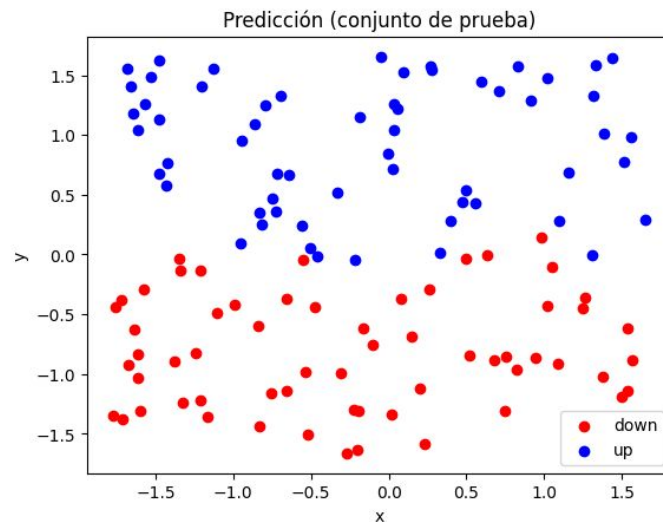
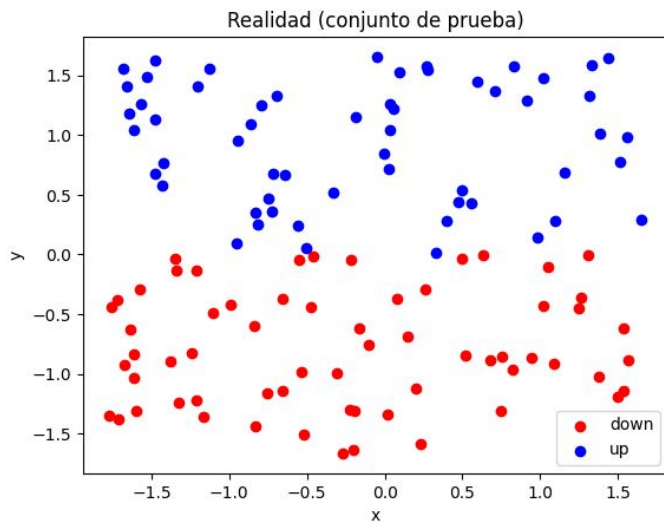
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Evaluación del desempeño

		Predicción	
		down	up
Realidad	down	60	3
	up	1	56



Variantes del algoritmo

- **Weighted k -NN (wkNN).** Pondera cada vecino por distancia ($w_i = 1/d$). Mientras más cercano esté el vecino al punto a clasificar, más pesa su clase. Reduce empates y el efecto de vecinos lejanos.
- **Radius Neighbors (ϵ -NN).** En lugar de utilizar k , utiliza un radio ϵ . Se usan todos los puntos dentro del radio. Útil cuando la densidad varía, pero puede dejar casos sin vecinos.
- **Adaptive k (local k -NN).** Elige k de forma local (según densidad o validación por punto) y evita usar la misma k en regiones densas y dispersas.

Conclusiones

- Es fácil explicar las decisiones del clasificador.
- El método funciona muy bien ante patrones caprichosos.
- Los datos deben escalarse para que el algoritmo funcione correctamente.
- El número de vecinos (k) queda a criterio de cada investigador. Un valor comúnmente usado es la raíz cuadrada del número de casos en el conjunto de entrenamiento.
- Como no hay un proceso de entrenamiento como tal, nunca se genera un modelo.
- A medida que k se incrementa, más cercano será el comportamiento al del clasificador ZeroR.

Recursos adicionales

- <https://sklearn.com/>
- <https://scikit-learn.org/>
- <https://developers.google.com/machine-learning/crash-course>